

```
#!/usr/bin/env python3
```

```
"""
```

Nanobot Personal AI Agent Harness & Housekeeper Daemon

Mined from: PLUGINS TOOLS AND MEMORY LAYER ARGUMENTS (File 7 & Marktechpost
Nanobot Notebook)

Role: Lightweight micro-agent container hooks, tool execution loop, and audit inspection hooks

Execution Target: Node Gamma (Rubik Pi 3 Housekeeper) / Node Alpha (Local Background
Watcher)

```
"""
```

```
import json
```

```
import inspect
```

```
from typing import Callable, Dict, Any, List, Optional
```

```
class Provider:
```

```
    """OpenAI-compatible inference client interface with fallback simulation."""
```

```
    def __init__(self, model: str = "qwen-2.5-0.5b-instruct", base_url: str =  
"http://localhost:20128/v1"):
```

```
        self.model = model
```

```
        self.base_url = base_url
```

```
    def complete(self, messages: List[Dict[str, str]], tools: Optional[List[Dict[str, Any]]] = None) ->  
Dict[str, Any]:
```

```
        # Connects via OmniRoute gateway on port 20128
```

```
        return {"role": "assistant", "content": "Nanobot execution verified."}
```

```
class ToolRegistry:
```

```
    """Registry mapping deterministic functions to tool JSON schemas."""
```

```
    def __init__(self):
```

```
        self.tools: Dict[str, Callable] = {}
```

```
        self.schemas: List[Dict[str, Any]] = []
```

```
    def register(self, func: Callable):
```

```
        sig = inspect.signature(func)
```

```
        doc = inspect.getdoc(func) or ""
```

```
        params = {}
```

```
        for name, param in sig.parameters.items():
```

```
            params[name] = {"type": "string", "description": f"Parameter {name}"}
```

```
        schema = {
```

```
            "type": "function",
```

```
            "function": {
```

```
                "name": func.__name__,
```

```

        "description": doc,
        "parameters": {
            "type": "object",
            "properties": params,
            "required": list(params.keys())
        }
    }
}
self.tools[func.__name__] = func
self.schemas.append(schema)
return func

```

class AuditHook:

```

    """Logs tool calls, arguments, and outcomes for Red Auditor telemetry."""
    def on_tool_start(self, tool_name: str, kwargs: Dict[str, Any]):
        print(f"[AUDIT] Executing tool: {tool_name} with params: {json.dumps(kwargs)}")

    def on_tool_end(self, tool_name: str, result: Any):
        print(f"[AUDIT] Tool {tool_name} completed. Result: {str(result)[:100]}")

```

class CensorHook:

```

    """Zero-trust sanitizer preventing raw tokens or forbidden paths from returning to agent."""
    FORBIDDEN_PATTERNS = ["rm -rf", "id_rsa", "eval(", "exec("]

    def filter_output(self, output: str) -> str:
        for pattern in self.FORBIDDEN_PATTERNS:
            if pattern in output:
                return "[REDACTED BY CENSOR HOOK: Security Invariant Violation]"
        return output

```

class NanobotAgent:

```

    def __init__(self, provider: Provider, registry: ToolRegistry):
        self.provider = provider
        self.registry = registry
        self.audit_hook = AuditHook()
        self.censor_hook = CensorHook()

    def run_step(self, prompt: str) -> str:
        self.audit_hook.on_tool_start("prompt_ingress", {"prompt": prompt})
        resp = self.provider.complete([{"role": "user", "content": prompt}], self.registry.schemas)
        clean_content = self.censor_hook.filter_output(resp.get("content", ""))
        self.audit_hook.on_tool_end("prompt_ingress", clean_content)
        return clean_content

```

```
if __name__ == "__main__":  
    registry = ToolRegistry()  
    @registry.register  
    def ping_housekeeper() -> str:  
        """Pings the Rubik Pi housekeeper daemon."""  
        return "PONG: Rubik Pi Housekeeper Active"  
  
    agent = NanobotAgent(Provider(), registry)  
    print(agent.run_step("Check housekeeper status"))
```